

UNIVERZITET U SARAJEVU
ELEKTROTEHNIČKI FAKULTET
ODSJEK ZA RAČUNARSTVO I INFORMATIKU

Algoritmi za višedimenzionalnu interpolaciju

ZAVRŠNI RAD

- PRVI CIKLUS STUDIJA -

Student:
Samra Salihić

Mentor:
Red. prof. dr Jurić Željko

Sarajevo,
juli 2019

Sažetak

U ovom radu dajemo opis, implementaciju i praktičnu upotrebu algoritama za višedimenzionalnu interpolaciju. Glavni dio rada daje opis algoritama višdimenzionalne iterpolacije te tretiranje dva različita slučaja ulaznih podataka, a zatim analiza za koje probleme je pogodno odabrati određeni algoritam.
Koncept ovog rada je takav da se prvo opisuje višedimenzionalna interpolacija, šta sa njom dobijamo i šta nam je potrebno na ulazu da bi mogli vršiti interpolaciju. Na osnovnu strukture podataka koju dobijamo na ulazu postoje dvije familije algoritama koje se dalje primjenjuju. Tako da se struktura rada razdvaja na interpaolaciju kada na ulazu imamo pravlino raspoređene podatke u mreži(eng. grid) i interpolaciju kada na ulazi imamo razbacane podatke u višedimenzijalnom prostoru. Za svaki algoritam je dat precizan opis i pseudokod, navedene prednosti i nedostaci primjene isth na određenu vrstu problema. Za neke od algoritme će biti prikazana implementacija u C++ programskom jeziku, te će rezultati biti upoređeni sa onim dobijenim u Matlabu.
Na kraju rada će biti opisano nekoliko slučajeva primjene višedimenzionalne interpolacije u rješavanju svakodnevnih problema u različitim naučnim oblastima.

Abstract

In this paper is given a comprehensive description, implementation and practical use of algorithms for multidimensional interpolation. The main part of the paper gives a description of the multidimensional iterpolation algorithms and the treatment two different cases of input data, and then analyzes for which problems it is appropriate to select a particular algorithm.
The concept of the thesis is such that it first describes multidimensional interpolation, what is result and usage of interpolation, and what we need at the input to be able to perform interpolation. Based on the data structure at the input there are two families of algorithms. The structure of the work is split into interpolation when data is in the grid and the interpolation when scattered data in the multi-dimensional space at the input. For each algorithm, a precise description and pseudocode is given, the advantages and disadvantages of applying algorithms to a specific type of problem. Few algorithms will be implemented in the C ++ programming language, and the results will be compared with those obtained in Matlab.
At the end of the paper will be described several use cases of applying multidimensional interpolation in solving ordinary problems in different scientific fields.

Elektrotehnički fakultet, Univerzitet u Sarajevu
Odsjek za računarstvo i informatiku
Red. prof. dr Jurić Željko
Sarajevo, ...(datum)

Postavka zadatka završnog rada I ciklusa:
Algoritmi za višedimenzionalnu interpolaciju

U okviru rada je potrebno razviti LaTeX predložak za izradu završnog rada prvog ciklusa studija. U radu je potrebno:

- objasniti opći postupak za izradu i pisanje završnih radova prvog ciklusa studija, poštivajući odgovarajuće Pravilnike,
- dati kratko uputstvo za korištenje .tex dokumenata, pisanje slika, tabela i relacija, generisanje sadržaja, popisa slika i tabela i sl.
- dati kratko upusto za generiranje odgovarajućih .pdf dokumenata iz odgovarajućih .tex fajlova,

Preporučuje se korištenje TexLive LaTeX podrške, verzije 2013 ili novije.

Napomena: U dijelu "Postavka" se stavlja postavka rada koju je specificirao mentor prilikom davanja teme. Naročito obratiti pažnju na naslov rada koji mora biti konzistentan tokom cijelog dokumenta, i usaglašen sa temom upisanom u informacioni sistem (npr. Zamger). Na ovoj stranici se potpisuje mentor prije nego se radovi odnesu u studentsku službu.

Polazna literatura:

[1] Sonka, M., Hlavec, V., Boyle, R., "Image Processing, Analysis, and Machine Vision", Thomson, 2008.

[2] Lee, E.A., Varaiya, P., "Structure and Interpretation of Signals and Systems", Electrical Engineering & Computer Science University of California, Berkeley, 2000

Red. prof. dr Jurić Željko

Univerzitet u Sarajevu
Elektrotehnički fakultet
Odsjek za računarstvo i informatiku

Izjava o autentičnosti radova

Završni rad
I ciklusa studija

Ime i prezime: Samra Salihić
Naslov rada: Algoritmi za višedimenzionalnu interpolaciju
Vrsta rada: Završni rad Prvog ciklusa studija
Broj stranica: ....

Potvrđujem:

- da sam pročitao dokumente koji se odnose na plagijarizam, kako je to definirano Statutom Univerziteta u Sarajevu, Etičkim kodeksom Univerziteta u Sarajevu i pravilima studiranja koja se odnose na I i II ciklus studija, integrirani studijski program I i II ciklusa i III ciklus studija na Univerzitetu u Sarajevu, kao i uputama o plagijarizmu navedenim na web stranici Univerziteta u Sarajevu;
- da sam svjestan univerzitetskih disciplinskih pravila koja se tiču plagijarizma;
- da je rad koji predajem potpuno moj, samostalni rad, osim u dijelovima gdje je to naznačeno;
- da rad nije predat, u cjelini ili djelimično, za stjecanje zvanja na Univerzitetu u Sarajevu ili nekoj drugoj visokoškolskoj ustanovi;
- da sam jasno naznačio prisustvo citiranog ili parafraziranog materijala i da sam se referirao na sve izvore;
- da sam dosljedno naveo korištene i citirane izvore ili bibliografiju po nekom od preporučenih stilova citiranja, sa navođenjem potpune reference koja obuhvata potpuni bibliografski opis korištenog i citiranog izvora;
- da sam odgovarajuće naznačio svaku pomoć koju sam dobio pored pomoći mentora i akademskih tutora/ica.

Sarajevo, datum ....

Potpis:

_____

Samra Salihić

Sadržaj

Popis slika **vi**

Popis tabela **vii**

1 Uvod **1**
1.1 Obrazloženje teme 1
1.2 Struktura rada 1

2 Višedimenzionalna interpolacija **2**
2.1 Dvodimenzionalna interpolacija u Matlabu 2

3 Interpolacija sa mrežom u više dimenzija **5**
3.1 Bilinearna interpolacija 5
3.2 Tačnost višeg stepena 6
3.3 Glatkoću većeg stepena: Bikubični splajn 7
3.4 Bikubična interpolacija (Glatkoća višeg reda) 8

4 Interpolacija razbacanih podataka u više dimenzija **12**
4.1 Interpolacija radijalne bazne funkcije (RBF) 12
4.2 Radijalne bazne funkcije u generalnoj upotrebi 14
4.3 Shepardova interpolacija 16
4.4 Interpolacija po Krigingu 17
4.5 Interpolacija krivih u više dimenzija 19

5 Primjena visedimenzionalne interpolacije **22**
5.1 Primjena višedimenzionalne interpolacije u procjeni performansi 22
5.1.1 Procjena podataka o gasu i nafti 22
5.2 Primjena multidimenzionalne interpolacije na nehomogene spužvaste kosti . . 22
5.3 Primjena višedimenzionalne interpolacije u procesiranju slike 22

Prilozi **24**

A Korištenje funkcija u Tex-u **25**
A.1 Matematički izraz 25
A.2 Slika 25
A.3 Tabela 25
A.4 *Landscape* 25
A.5 Indeks pojmova i Popis oznaka 26
A.6 Korištenje literature 26

A.7 Programski kodovi 26

Literatura **28**

Indeks pojmova **28**

Popis slika

2.1 Formiranje vektora tačaka u kojima su poznate vrijednosti funkcije 3
2.2 Formiranje matrice poznatih vrijednosti funkcije 3
2.3 Garf funkcije - Metoda interpolacije 'linear' 4
2.4 Garf funkcije - Metoda interpolacije 'spline' 4

Popis tabela

A.1 Naslov tabele 26

POPIS TABELA *Tesla, N., "Predložak za izradu završnog rada"* viii

Poglavlje 1

Uvod

U skladu sa dobrom istraživačkom praksom, uvodno poglavlje rada prvog ciklusa studija bi trebao sadržavati bar sljedeće elemente:

- obrazloženje teme,
- opis strukture rada.

U narednom tekstu će detaljnije biti obrazložena svaka od tačaka.

1.1 Obrazloženje teme

U ovoj sekciji autor je dužan da obrazloži koja tema ili problem će biti analizirani ili istraživani, te zbog čega je upravo ova tema pogodna i bitna za istraživanje. Pohvalno je napraviti pregled literature sa odgovarajućim referenciranjem na istu.

1.2 Struktura rada

U ovoj sekciji je najbolje dati po jedan paragraf o svakom poglavlju iz rada. Potencirajte koji su glavni doprinosi svakog poglavlja, te kako su poglavlja medjusobno povezana.

Poglavlje 2

Višedimenzionalna interpolacija

Interpolacija u oblasti numeričke matematike predstavlja metod izračunavanja vrijednosti funkcije u novim tačkama koje se nalaze unutar intervala definiranog diskretnim skupom podataka. Sam pojam višedimenzionalna interpolacija dolazi iz osobina funkcije koju interpoliramo. Npr. ukoliko se interpolira funkcija dvije promijenljive onda je riječ o dvodimenzionalnoj interpolaciji, itd.
U višedimenzionalnoj interpolaciji tražimo procjenu funkcije više od jedne nezavisne varijable $y(x_1, x_2, ..., x_n)$. Postoje dva pristupa višedimenzionalnoj interpolaciji, a koji ćemo od ova dva pristupa izabrati zavisi od podataka koji su dati na ulazu. Moguće je da su poznate vrijednosti funkcije u tačkama koje formiraju mrežu (engl. grid), ili da su poznate vrijednosti funkcije u slučajnim tačkama u n-dimenzionalnom prostoru.
U jednodimenzionalnoj interpolaciji to pitanje se nikada nije pojavilo, jer bilo koji skup x_i, jednom sortiran u rastućem redoslijedu, može se promatrati kao valjana jednodimenzionalna mreža (regularni razmak između poznatih vrijednosti funkcije nije uslov).
Kako se broj dimenzija n povećava, održavanje pune mreže postaje nepraktično, zbog ekspanzije broja tačaka na mreži. Obje metode su praktične u dvodimenzionalnom prostoru.
U nastavku poglavlja bit će prikazan primjer dvodimenzionalne interpolacije u Matlabu, sa ciljem da shvatanje poglavlja 4 i 3 bude jednostavnije.

2.1 Dvodimenzionalna interpolacija u Matlabu

Kao što je prethodno rečeno, na ulazu su potrebne vrijednosti funkcije i tačke na kojim su te vrijednosti poznate.
U prvom primjeri pravimo mrežu tačaka i poznatih vrijednosti u tim tačkama a zatim interpoliramo i prikazujemo graf te funkcije. Na slikama 2.1 i 2.2 je prikazano pravljenje mreže poznatih vrijednosti u Matlabu.

```
[X1,X2] = ndgrid((-5:1:5))
```

```
X1 = 11×11
    -5    -5    -5    -5    -5    -5    -5    -5    -5    -5    -5
    -4    -4    -4    -4    -4    -4    -4    -4    -4    -4    -4
    -3    -3    -3    -3    -3    -3    -3    -3    -3    -3    -3
    -2    -2    -2    -2    -2    -2    -2    -2    -2    -2    -2
    -1    -1    -1    -1    -1    -1    -1    -1    -1    -1    -1
     0     0     0     0     0     0     0     0     0     0     0
     1     1     1     1     1     1     1     1     1     1     1
     2     2     2     2     2     2     2     2     2     2     2
     3     3     3     3     3     3     3     3     3     3     3
     4     4     4     4     4     4     4     4     4     4     4
     ⋮

X2 = 11×11
    -5    -4    -3    -2    -1     0     1     2     3     4     5
    -5    -4    -3    -2    -1     0     1     2     3     4     5
    -5    -4    -3    -2    -1     0     1     2     3     4     5
    -5    -4    -3    -2    -1     0     1     2     3     4     5
    -5    -4    -3    -2    -1     0     1     2     3     4     5
    -5    -4    -3    -2    -1     0     1     2     3     4     5
    -5    -4    -3    -2    -1     0     1     2     3     4     5
    -5    -4    -3    -2    -1     0     1     2     3     4     5
    -5    -4    -3    -2    -1     0     1     2     3     4     5
    -5    -4    -3    -2    -1     0     1     2     3     4     5
     ⋮
```

Slika 2.1: Formiranje vektora tačaka u kojima su poznate vrijednosti funkcije

```
R = sqrt(X1.^2 + X2.^2)+ eps;
V = sin(R)./(R)
```

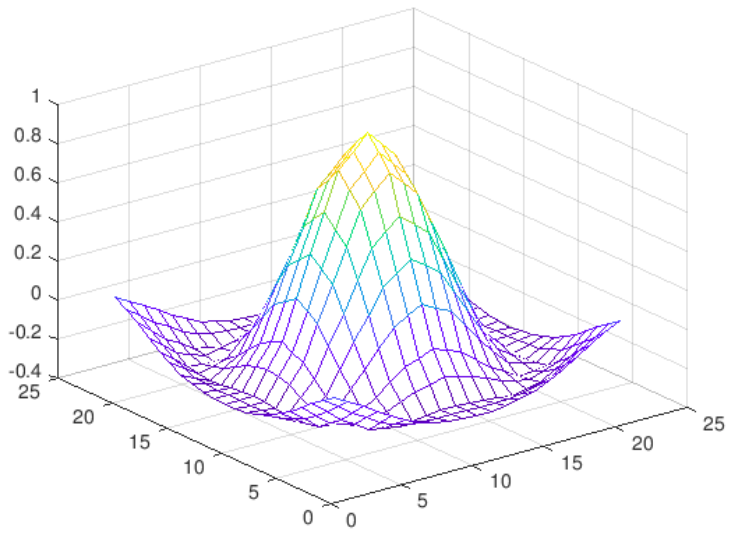
```
V = 11×11
    0.1002    0.0187   -0.0749   -0.1452   -0.1816   -0.1918   -0.1816   -0.1452   -0.0749    0.0187    0.1002
    0.0187   -0.1036   -0.1918   -0.2172   -0.2016   -0.1892   -0.2016   -0.2172   -0.1918   -0.1036    0.0187
   -0.0749   -0.1918   -0.2102   -0.1241   -0.0065    0.0470   -0.0065   -0.1241   -0.2102   -0.1918   -0.0749
   -0.1452   -0.2172   -0.1241    0.1089    0.3518    0.4546    0.3518    0.1089   -0.1241   -0.2172   -0.1452
   -0.1816   -0.2016   -0.0065    0.3518    0.6985    0.8415    0.6985    0.3518   -0.0065   -0.2016   -0.1816
   -0.1918   -0.1892    0.0470    0.4546    0.8415    1.0000    0.8415    0.4546    0.0470   -0.1892   -0.1918
   -0.1816   -0.2016   -0.0065    0.3518    0.6985    0.8415    0.6985    0.3518   -0.0065   -0.2016   -0.1816
   -0.1452   -0.2172   -0.1241    0.1089    0.3518    0.4546    0.3518    0.1089   -0.1241   -0.2172   -0.1452
   -0.0749   -0.1918   -0.2102   -0.1241   -0.0065    0.0470   -0.0065   -0.1241   -0.2102   -0.1918   -0.0749
    0.0187   -0.1036   -0.1918   -0.2172   -0.2016   -0.1892   -0.2016   -0.2172   -0.1918   -0.1036    0.0187
     ⋮
```

Slika 2.2: Formiranje matrice poznatih vrijednosti funkcije

Nakon toga interpoliramo funkciju tako što radimo interpolaciju na polovinama intervala u po svakoj dimenziji izmedju svih definisanih tačaka i zatim se prikazuje kako graf funkcije izgleda sa svim tačkama.
Na slici 2.3 je prikazan graf kada interpoliramo prethodnu funkciju metodom *linear*. Kod je dat u nastavku.

```
1 Vq = interpn(V,'linear');
2 mesh(Vq);
```

Program 2.1: Linearna interpolacija

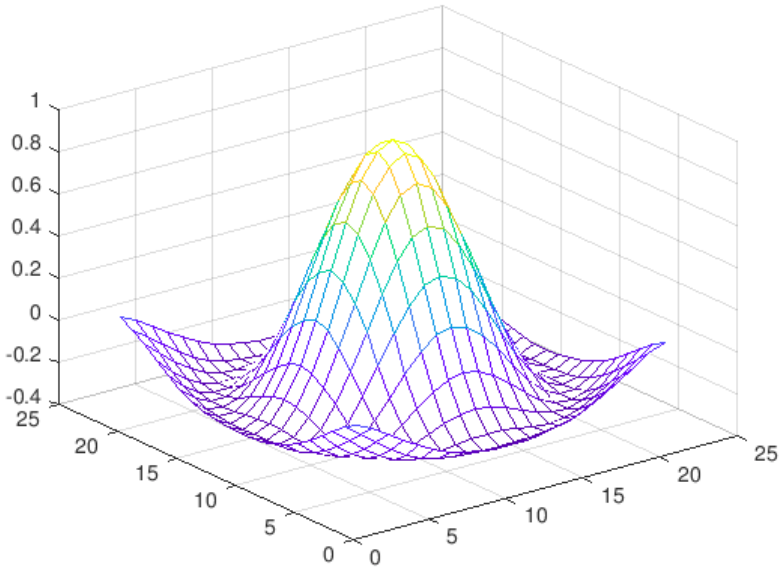


Slika 2.3: Garf funkcije - Metoda interpolacije 'linear'

Na slici 2.4 je prikazan graf kada interpoliramo prethodnu funkciju metodom *spline*. Kod je dat u nastavku.

```
Vq = interpn(V,'spline');
mesh(Vq);
```

Program 2.2: Splajn interpolacija



Slika 2.4: Garf funkcije - Metoda interpolacije 'spline'

Može se primjetiti da primjenom različitih metoda interpolacije se dobijaju različiti grafici funkcije. U prethodnim primjerima vidimo da je grafik funkcije interpolirane metodom koja koristi splajnove "glađi", nego grafik dobijen linearnom metodom interpolacije.
U nastavku rada bit će biti opisani algoritmi za ovakve metode interpolacije.

Poglavlje 3

Interpolacija sa mrežom u više dimenzija

Zbog boljeg razumijevanja ovog poglavlja algoritmi koji će biti razmatrani u nastavku odnose se na dvodimenzionalnu interpolaciju (što spada u višedimenzionalnu interpolaciju). Sve navedene metode generaliziraju se na tri dimenzije na očigledan način. Kada se implementiraju metode za razbacane podatke, u poglavlju 4, tretman će biti isti za opće n. U ostatku ovog odjeljka razmatran je samo slučaj interpolacije rešetke, a u kodu je implementiran (najčešći) slučaj sa dvije dimenzije.

U dvije dimenzije posmatrana je matrica vrijednosti funkcije f_{ij}, gdje $i = 0,...,M-1$ i $j = 0,...,N-1$. Također, poznat je vektor x_1 sa vrijednostima x_{1i} i vektor x_2 sa vrijednostima x_{2j}. Veza između spomenutih parametara je

$$f_{ij} = y(x_{1i}, x_{2j})$$

Postavlja se pitanje, šta je potrebno da interpolacija omogući? Potrebno je procijeniti vrijednosti funkcije u tački (x_1, x_2) čija vrijednosti nije poznata u matrici(mreži).
Prvi korak je pronaći kvadrat u mreži koji omeđuje traženu tačku (x_1, x_2). To su ustvari 4 tačke koje zadovoljavaju sljedeće uslove:

$$x_{1i} \leq x_1 \leq x_{1(i+1)}$$

$$x_{2j} \leq x_2 \leq x_{2(j+1)}$$

Kako ove relacije definišu konkretne vrijednosti i i j, onda:

$$y_0 \equiv y_{ij}$$

$$y_1 \equiv y_{(i+1)j}$$

$$y_2 \equiv y_{(i+1)(j+1)x}$$

$$y_3 \equiv y_{i(j+1)}$$

Sada je potrebno odlučiti se koji algoritam koristiti za interpolaciju kada je pronađen kvadrat u mreži. U nastavku će za početak biti objašnjen najjednostavniji metod za dvodimenzionalnu interpolaciju a to je *bilinearna interpolacija* na kvadratnoj rešetki.

3.1 Bilinearna interpolacija

Kako su već poznate vrijednosti 4 tačke koje čine kvadrat može se preći na forumule koje koristi bilinearna interpolacija:

$$t \equiv (x_1 - x_{1i})/(x_{1(i+1)} - x_{1i})$$

$$u \equiv (x_2 - x_{2j})/(x_{2(j+1)} - x_{2j})$$

tako da se vrijednost t i u nalazi u opsegu [0,1], i forumla

$$y_{(x_1,x_2)} = (1-t)(1-u)y_0 + t(1-u)y_1 + tuy_2 + (1-t)uy_3$$

Kada se interpolira više tačaka potrebno je za svaku tačku tražiti kvadrat koji je okružuje što se nekad može svesti na to da se par iteracija nalazi u istom kvadratu. Iz tog razloga, da se smanji vrijeme izvršavanja i bespotrebnih provjera moguće je pamtit prethodno pronađene kvadrate. Implementacija bilinearne interpolacije u C++ je prikazana u nastavku.

```
class Bilin_interp {
    int m,n;
    const vector<vector<double> > &y;
    Linear_interp x1terp, x2terp;
public:
    Bilin_interp(vector<double> &x1v, vector<double> &x2v, vector<
    vector<double> > &ym): m(x1v.size()), n(x2v.size()), y(ym),
    x1terp(x1v,x1v), x2terp(x2v,x2v) {}

    double interp(double x1p, double x2p) {
        int i,j;
        double yy, t, u;
        i = x1terp.cor ? x1terp.hunt(x1p) : x1terp.locate(x1p);
        j = x2terp.cor ? x2terp.hunt(x2p) : x2terp.locate(x2p);
        t = (x1p-x1terp.xx[i])/(x1terp.xx[i+1]-x1terp.xx[i]);
        u = (x2p-x2terp.xx[j])/(x2terp.xx[j+1]-x2terp.xx[j]);
        yy = (1.-t)*(1.-u)*y[i][j] + t*(1.-u)*y[i+1][j] + (1.-t)*u*y
    [i][j+1] + t*u*y[i+1][j+1];
        return yy;
    }
};
```

Program 3.1: Bilinearna interpolacija - C++ kod

Klasa *BilinInterp* kao svoje atribute ima matricu y, i dva vektora: x1terp i x2terp. Ovako dva vektora su objekti klase *LinearInterp* iz razloga da bi se koristile metode koje su implementirane za jednodimenzionalnu interpolaciju, a to su u ovom primjeru metode *locate* i *hunt*. Ove metode služe za prethodno spomenut kvadrat koji omeđuje tačku za koju vršimo interpolaciju. Bilinearna interpolacija je samo ekstenzija linearnoj interpolaciji za interpoliranje funkcija sa dvije promijenjive.

Kada se uporede rezultati dobiveni u Matlabu korištenjem metode *linear* i rezultati dobiveni pozivom metode *interp* nad objektom klase *BilinInterp*, dobijeni rezultati budu jednaki.

Bilinearna interpolacija je dobra za početak, osim ako nije pozato da je potrebno više. Postoje dva izrazito različita smjera koja se mogu uzeti pored bilinearne interpolacije za metode višeg poretka: može se koristiti viši poredak za dobivanje bolje točnosti interpolirane funkcije (za dovoljno glatke funkcije!), bez nužnog pokušaja da se popravi kontinuitet gradijenta i viših derivata. Ili drugi način, može se koristiti viši poredak kako bi se nametnula glatkoća nekih tačaka i izvodi kao kada interpolacijska tačka prelazi granice kvadrata. Sada će biti razmotren svaki od ova dva načina.

3.2 Tačnost višeg stepena

Osnovna ideja metoda je razbiti problem u niz jednodimenzionalnih interpolacija. Ako se želi napraviti interpolacija m-1 reda u smjeru x_1, i n-1 reda u smjeru x_2, najprije treba locirati *mxn*

kvadrat kojim se locira tražena tačka (x_1, x_2). Tada treba učiniti jednodimenzionalnu interpolaciju u smjeru x2, tj. u redovima pronađenog kvadrata, da bi se dobile vrijednosti funkcije u tačkama (x_{1i}, x_2), sa m vrijednosti i. Na kraju, koristeći prethodnu metodu jednodimenzionalne interpolacije pravi se finalna interpolacija ali sada u smjeru x_1 da bi se dobio konačan rezultat. Implementacija polinomijalne interpolacije u C++ je prikazana u nastavku.

```
1
2  class Poly2D_interp {
3      int m,n,mm,nn;
4      const vector<vector<double> > &y;
5      vector<double> yv;
6      Poly_interp x1terp, x2terp;
7  public:
8      Poly2D_interp(vector<double> &x1v, vector<double> &x2v, vector<
       vector<double> > &ym, int mp, int np) : m(x1v.size()), n(x2v.
       size()),
9          mm(mp), nn(np), y(ym), yv(m),
10         x1terp(x1v,yv,mm), x2terp(x2v,x2v,nn) {}
11
12     double interp(double x1p, double x2p) {
13         int i,j,k;
14         i = x1terp.cor ? x1terp.hunt(x1p) : x1terp.locate(x1p);
15         j = x2terp.cor ? x2terp.hunt(x2p) : x2terp.locate(x2p);
16         for (k=i;k<i+mm;k++) {
17             x2terp.yy = &y[k][0];
18             yv[k] = x2terp.rawinterp(j,x2p);
19         }
20         return x1terp.rawinterp(i,x1p);
21     }
22 };
```

Program 3.2: Polinomijalna 2D interpolacija - C++ kod

Kao što je prethodno rečeno ovaj problem razbijamo u niz jednodimenzionalnih interpolacija. Koristimo polinomijalnu interpolaciju za interpolaciju funkcija sa jednom promijenjivom. Vidimo da u metodi *interp* ponovo tražimo kvadrat koji omeđuje tačku koju želimo interpolirati. Nakon toga radimo mm (prethodno definisana vrijednost) interpolacija u smjeru x2, a zatim finalnu interpolaciju u smjeru x1 kako bi dobili konačni rezultat.

Kada se uporede rezultati dobiveni u Matlabu korištenjem metode *makima* (koja u implementaciji koristi polinome) i rezultati dobiveni pozivom metode *interp* nad objektom klase *Poly2dInterp*, dobijeni rezultati budu jednaki.

3.3 Glatkoću većeg stepena: Bikubični splajn

Osnovna tehnika za postizanje glatkoće u dvodimenzionalnoj interpolaciji je bikubični splajn. Da bi se postavio bikubični spline, potrebno je jednom konstruisati M jednodimenzionalnih splajnova preko redova dvodimenzionalne matrice koja predstavlja vrijednosti funkcije. Zatim, za svaku željenu interpoliranu vrijednost postupiti na sljedeći način:

1. Koristiti splajn interpolaciju M puta, kako bi se dobio vektor vrijednosti $y(x_{1i}, x_2), i = 0, \ldots, M-1$
2. Konstruisati jednodimenzionalni splajn kroz te vrijednosti.
3. Interpolirati splajn interpolacijom do tražene tačke (x_1, x_2)

Podešavanje koje se treba samo jednom uraditi se povećava sa veličinom matrice *MxN* , dok se izračunavanje po interpolaciji skalira ugrubo kao $MlogM + N$ sa jakim konstantama ispred koje se ne prikazuju. To je cijena koja se plaća za željenje karakterisike. Razlog tome leži u njihovoj nelokalnosti. Za matrice *MxN* čija veličina je manja od par stotina cijena koja se plaća je tolerantna. Ukoliko se radi o većim matricama, bolje je uzeti neku od prethodno spomenutih metoda.

```
1
2  class Spline2D_interp {
3      int m,n;
4      const vector<vector<double> > &y;
5      vector<double> &x1;
6      vector<double> yv;
7      vector<Spline_interp*> srp;
8  public:
9      Spline2D_interp(vector<double> &x1v, vector<double> &x2v, vector
       <vector<double> > &ym)
10         : m(x1v.size()), n(x2v.size()), y(ym), yv(m), x1(x1v), srp(m
       ) {
11         for (int i=0;i<m;i++) srp[i] = new Spline_interp(x2v,&y[i
       ][0]);
12     }
13
14     ~Spline2D_interp(){
15         for (int i=0;i<m;i++) delete srp[i];
16     }
17
18     double interp(double x1p, double x2p) {
19         for (int i=0;i<m;i++) yv[i] = (*srp[i]).interp(x2p);
20         Spline_interp scol(x1,yv);
21         return scol.interp(x1p);
22     }
23 };
```

Program 3.3: Splajn 2D interpolacija - C++ kod

Slično kao i u prethodno opisanim metodama, i ovdje se koristi metoda jednodimenzionalne interpolacije. Da bi postigli veću glatkoću funkcije koju interpoliramo koristimo splajnove. Konkretno kako se može iskoristiti interpolacija splajnovima je prikazano u kodu iznad. Koristeći splajn interpolaciju sa podacima iz vektora x2, interpoliramo splajnom koordinatu tačke x2p m puta. Time pravimo vektor vrijednsoti yv, tj. jednodimenzionalni splajn kroz te vrijednosti. Njega dalje koristimo da napravimo finalnu interpolaciju splajnom ali sara koristeći vektor x1 i prethodno dobijeni jednodimenzionalni splajn yv. Finalna interpolacija se radi u x1p kooradinati tačke.

Kada se uporede rezultati dobiveni u Matlabu korištenjem metode *spline* (koja u implementaciji koristi polinome) i rezultati dobiveni pozivom metode *interp* nad objektom klase *Spline2dInterp*, dobijeni rezultati budu jednaki.

3.4 Bikubična interpolacija (Glatkoća višeg reda)

Bikubična interpolacija ima za rezultat isti stepen glatkoće kao i interpolacija bikubičnim splajnom, prednost bikubične interpolacije ogleda se u tome što je lokalna metoda. Pa zbog toga nakon postavke izračunavanje interpolacije je konstanta, log M + log N, za pronalazak mjesta

u tabeli. Ova prednost dosta usložnjava kodiranje. U nastavku su dati samo neki od gradivnih elementa za metodu, ali ne i potpuni korisnički interfejs.

Bikubični splajnovi su ustvari specijalan slučaj bikubične interpolacije. U općem slučaju vrijednosti svih izvoda na tačkama mreže su proizvoljno određene. Mogu se specificirati na bilo koji način. Odnosno, na svakoj tački mreže se osim funkcije $y(x_{1i},x_2)$ odredjuje i gradijent $\partial y/\partial x_1 \equiv y_{,1}$, $\partial y/\partial x_2 \equiv y_{,2}$ i parcijalni izvodi $\partial^2 y/\partial x_1 \partial x_2 \equiv y_{,12}$. Tada se može pronaći interpolacijska funkcija koja je 'kubična' u skaliranim koordinatama t i u, sa sljedećim svojstvima: (i) Vrijednosti funkcije i navedenih izvoda reproduciraju se tačno na tačkama mreže, i (ii) vrijednosti funkcije i specificiranih derivata kontinuirano se mijenjaju dok interpolacijska tačka prelazi iz jednog kvadrata mreže u drugi.

Važno je razumjeti da se u jednačinama bikubične interpolacije ne zahtijeva da tačno budu određeni dodatni derivati. Poseban problem je kako dobiti navedene vrijednosti. Bolje vrijednosti znače precizniju interpolaciju.

Najbolji način je analitički znati izvode ili ih moći precizno izračunati numeričkim putem na tačkama mreže. Sljedeći najbolji način je odrediti ih numeričkim diferenciranjem od vrijednosti funkcije koje su već tabelirane na mreži. Relevantni kôd bi bio nešto slično ovome (pomoću središnjeg diferenciranja)

```
y1a[j][k]=(ya[j+1][k]-ya[j-1][k])/(x1a[j+1]-x1a[j-1]);
y2a[j][k]=(ya[j][k+1]-ya[j][k-1])/(x2a[k+1]-x2a[k-1]);
y12a[j][k]=(ya[j+1][k+1]-ya[j+1][k-1]-ya[j-1][k+1]+ya[j-1][k-1])
/((x1a[j+1]-x1a[j-1])*(x2a[k+1]-x2a[k-1]));
```

Program 3.4: Splajn interpolacija

Da bi se uradila bikubična interpolacija unutar kvadratne mreže, sa zadatom funkcijom y i izvodima y_1, y_2, yy_{12} na svakoj od četiri ivice kvadrata, prave se dva koraka: U prvom koraku se dobija 16 veličina $c_{ij}, i,j = 0,...,3$ koristeći proceduru *bcucof* navedenu ispod. (Formule za dobivanje vrijednosti c iz funkcije i izvedenih vrijednosti su poprilično komplicirane, ali nakon što se jednom odrede vrijednosti, one se mogu tabelirati i zaboraviti.) Dalje, zamijene se c-ovi sa bilo kojom ili sa svim sljedećim formulama za funkciju i izvode, po želji:

$$y(x_1,x_2) = \sum_{i=0}^{3}\sum_{j=0}^{3} c_{ij}t^i u^j$$

$$y_{,1}(x_1,x_2) = \sum_{i=0}^{3}\sum_{j=0}^{3} i c_{ij}t^{i-1}u^j (dt/dx_1)$$

$$y_{,2}(x_1,x_2) = \sum_{i=0}^{3}\sum_{j=0}^{3} j c_{ij}t^i u^{j-1} (dt/dx_2)$$

$$y_{,12}(x_1,x_2) = \sum_{i=0}^{3}\sum_{j=0}^{3} ij c_{ij}t^{i-1}u^{j-1} (dt/dx_1)(dt/dx_2)$$

gdje su t i u dati jednačinom (3.6.4).

```

void bcucof(vector<double> &y, vector<double> &y1, vector<double> &
    y2, vector<double> &y12,
    const double d1, const double d2, vector<vector<double> > &c) {
    static int wt_d[16*16]=
```

```
        {1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
        -3, 0, 0, 3, 0, 0, 0, 0,-2, 0, 0,-1, 0, 0, 0, 0,
        2, 0, 0,-2, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0,
        0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0,
        0, 0, 0, 0,-3, 0, 0, 3, 0, 0, 0, 0,-2, 0, 0,-1,
        0, 0, 0, 0, 2, 0, 0,-2, 0, 0, 0, 0, 1, 0, 0, 1,
        -3, 3, 0, 0,-2,-1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0,-3, 3, 0, 0,-2,-1, 0, 0,
        9,-9, 9,-9, 6, 3,-3,-6, 6,-6,-3, 3, 4, 2, 1, 2,
        -6, 6,-6, 6,-4,-2, 2, 4,-3, 3, 3,-3,-2,-1,-1,-2,
        2,-2, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 2,-2, 0, 0, 1, 1, 0, 0,
        -6, 6,-6, 6,-3,-3, 3, 3,-4, 4, 2,-2,-2,-2,-1,-1,
        4,-4, 4,-4, 2, 2,-2,-2, 2,-2,-2, 2, 1, 1, 1, 1};
    int l,k,j,i;
    double xx,d1d2=d1*d2;
    vector<double> cl(16),x(16);
    //static MatInt wt(16,16,wt_d);
    vector<vector<int>> wt(16, vector<int>(16));
    int *a = wt_d;
    for (i=0; i<16; i++) for (j=0; j<16; j++) wt[i][j] = *a++;
    for (i=0;i<4;i++) {
        x[i]=y[i];
        x[i+4]=y1[i]*d1;
        x[i+8]=y2[i]*d2;
        x[i+12]=y12[i]*d1d2;
    }
    for (i=0;i<16;i++) {
        xx=0.0;
        for (k=0;k<16;k++) xx += wt[i][k]*x[k];
        cl[i]=xx;
    }
    l=0;
    for (i=0;i<4;i++)
        for (j=0;j<4;j++) c[i][j]=cl[l++];
}
```

Program 3.5: Primjer programa

Implementacija jednačine (3.6.6), koja izvodi bikubičnu interpolaciju, vraća interpoliranu vrijednost i dvije vrijednosti gradijenta i koristi proceduru *bcucof*, pa je jednostavno:

```

void bcuint(vector<double> &y, vector<double> &y1, vector<double> &
    y2, vector<double> &y12,
    const double x1l, const double x1u, const double x2l, const
    double x2u,
    const double x1, const double x2, double &ansy, double &ansy1,
    double &ansy2) {
    int i;
    double t,u,d1=x1u-x1l,d2=x2u-x2l;
    vector<vector<double> > c(4, vector<double>(4));
    bcucof(y,y1,y2,y12,d1,d2,c);
    if (x1u == x1l || x2u == x2l)
        throw("Bad input in routine bcuint");
    t=(x1-x1l)/d1;
    u=(x2-x2l)/d2;
```

```
13      ansy=ansy2=ansy1=0.0;
14      for (i=3;i>=0;i--) {
15          ansy=t*ansy+((c[i][3]*u+c[i][2])*u+c[i][1])*u+c[i][0];
16          ansy2=t*ansy2+(3.0*c[i][3]*u+2.0*c[i][2])*u+c[i][1];
17          ansy1=u*ansy1+(3.0*c[3][i]*t+2.0*c[2][i])*t+c[1][i];
18      }
19      ansy1 /= d1;
20      ansy2 /= d2;
21  }
```

Program 3.6: Primjer programa

Mogu se kombinirati najbolje osobine bikubične interpolacije i bikubičnih splajnova tako što bi se splajnovi koristili za izračunavanje vrijednosti potrebnih izvoda u tačkama mreže, zatim pohranjivale te vrijednosti, a nakon toga upotrebljavala bikubična interpolacije uz učinkoviti metod pretraživanja tablica, za interpolaciju stvarnih funkcija.

Poglavlje 4

Interpolacija razbacanih podataka u više dimenzija

Poslije definisanih interpolacija na pravilnom obliku rešetke se prelazi na mnogo zahtjevniju interpolaciju skupa raspršenih tačaka u višedimezionalnom prostoru. Dat je proizvoljno raspršeni skup od N tačaka $(x_i, j_i), i = 0, ..., N-1$ u n-dimenzionalnom prostoru. Gdje je sa x_i označen n-dimenzionalni vektor nezavisnih varijabli, $(x_{1i}, x_{2i}, ..., x_{ni})$, dok je y_i vrijednost funkcije u odabranoj tački.

Razmatrana su dva najšire korištena opšta metoda za rješavanje ovog tipa problema, interpolacija radijalne bazne funkcije (RBF) i Kriging metoda. Obje metode su računski zahtjevne. Ove metode zahtijevaju operacije $O(N^3)$ da bi na početku sistematizirali skup podataka, za svaku interpolaciju vrijednosti potrebno je O(N) operacija. Kriging metodom mozemo procijeniti greške interpolacije po prilično visokoj cijeni izračunavanja $O(N^3)$ za svaku interpoliranu vrijednost. Shepardova interpolacija je varijanta RBF-a koja izbjegava početno izračunavanje koje efektivno ograničava interpolaciju za sve skupove čije je N približno ili veće od 10000 tačaka. Pa se isplati razmotriti i druge opcije. Neke od opcija su:

- Ako n nije prevelik (obično n = 2), i ako su tačke podataka poprilično guste, te zatim razmotriti triangulaciju. Triangulacija je primjer metode konačnih elemenata. Takve metode konstruišu neki privid geometrijske regularnosti, a zatim koristi to pri konstrukciji kao prednost. Kreiranje mreže tačaka je usko povezana tema.

- Ako tačnost izračunavanja to toleriše, moguće je premještanje svake tačke podataka u najbliže tačke na regularnoj kartezijanskoj mreži, a zatim pomoću Laplaceove interpolacije popunjavaju se preostale tačake na mreži. Kao i obično potrebno je napraviti kompromis između granulacije mreže (kako bi se smanjila greška uvedena micanjem tačaka) i računskog vremena utrošenog Laplaceovom metodom.

Ako nijedna od ovih opcija nije dobra za rješenje zadatog problema, preostaju 'klasične' metode, RBF i Kriging. Koja metoda bolje funkcionira zavisi od pojedinosti problema.

4.1 Interpolacija radijalne bazne funkcije (RBF)

Ideja interpolacije RBF-a je vrlo jednostavna: zamislite da svaka poznata tačka j "utiče" na svoje okruženje na isti način u svim smjerovima, prema nekim pretpostavljenim funkcionalnim oblicima $\phi(r)$ (radialno bazna funkcija). To je funkcija radijalne udaljenosti od tačke $r = |x - x_j|$

. Aproksimacija funkcije interpolacije linearnom kombinacijom vrijednosti $\phi(r)$, centriranom na sve poznate tačke,

$$y(x) = \sum_{i=0}^{N-1} \omega_i \phi(|x - x_i|)$$

gde je ω_i skup nepoznatih težina. Te težine se određuju preko vrijednosti funkcije y_i. Težine se određuju tako što se zahtijeva da interpolacija bude tačna na svim poznatim tačkama. Što je ekvivalentno rješavanju skupa N linearnih jednačina sa N nepoznatih za ω_i:

$$y_j = \sum_{i=0}^{N-1} \omega_i \phi(|x_J - x_i|)$$

Za mnoge funkcionalne forme, može se dokazati, pod različitim opštim pretpostavkama, da je ovaj skup jednadžbi nedegenerisan i može se lako riješiti, npr. LU dekompozicijom. Varijanta interpolacije RBF-a je normalizirana interpolacija radijalne bazne funkcije (NRBF), u kojoj je potrebno da suma osnovnih funkcija bude ekvivalentna pa zamijenom predhodnih jednačina dobija se:

$$y(x) = \sum_{i=0}^{N-1} \omega_i \phi(|x - x_i|) / \sum_{i=0}^{N-1} \phi(|x - x_i|)$$

Prethodne jednačine nastaju prirodnije iz Bayesove statističke perspektive. Nema dokaza da je NRBF metoda konzistentno bolja od RBF metode, ili obrnuto. Lako je implementirati oba metoda u istom kodu.

Za N tačaka početno odredjivanje težina vrši se pomoću LU dekompozicije za $O(N^3)$. Nakon toga za svaku interpolaciju O(N). Prema tome, N = 1000 je gruba linija razdvajanja između "lakog" i "teškog".

Evo nekoliko objekata koji implementiraju sve o čemu se raspravljalo do sada. *RBF_fn* je virtualna bazna klasa čije izvedene klase će predstavljati različite funkcionalne forme za $rbf(r) \equiv \phi(r)$. *RBF_interp*, preko konstruktora, sistematizira podatke i rješava jednačine za zadate vrijednosti. Tačke x_i unose se kao N x n matrica, a kod radi za bilo koju dimenziju n. Boolean ulazni argument *nrbf* govori da li će se koristiti NRBF umjesto RBF. Funkcija interp se poziva za dobijanje interpolirane funkcijske vrijednosti u novoj tački x.

```
1
2 class RBF_interp {
3     int dim, n;
4     const vector<vector<double> > &pts;
5     const vector<double> &vals;
6     vector<double> w;
7     RBF_fn &fn;
8     bool norm;
9 public:
```

```
10      RBF_interp(vector<vector<double> > &ptss, vector<double> &valss,
        RBF_fn &func, bool nrbf=false): dim(ptss[0].size()), n(ptss.
    size()) , pts(ptss), vals(valss),
11      w(n), fn(func), norm(nrbf) {
12          int i,j;
13          double sum;
14          vector<vector<double> > rbf(n);
15          for(int i = 0; i < rbf.size(); i++) rbf[i].resize(n);
16          vector<double> rhs(n);
17          for (i=0;i<n;i++) {
18              sum = 0.;
19              for (j=0;j<n;j++) {
20                  sum += (rbf[i][j] = fn.rbf(rad(&pts[i][0],&pts[j
    ][0])));
21              }
22              if (norm) rhs[i] = sum*vals[i];
23              else rhs[i] = vals[i];
24          }
25          LUdcmp lu(rbf);
26          lu.solve(rhs,w);
27      }
28
29      double interp(vector<double> &pt) {
30          double fval, sum=0., sumw=0.;
31          if (pt.size() != dim) throw("RBF_interp bad pt size");
32          for (int i=0; i<n; i++) {
33              fval = fn.rbf(rad(&pt[0],&pts[i][0]));
34              sumw += w[i]*fval;
35              sum += fval;
36          }
37          return norm ? sumw/sum : sumw;
38      }
39
40      double rad(const double *p1, const double *p2) {
41          double sum = 0.;
42          for (int i=0;i<dim;i++) sum += SQR(p1[i]-p2[i]);
43          return sqrt(sum);
44      }
45  };
```

Program 4.1: Splajn 2D interpolacija - C++ kod

4.2 Radijalne bazne funkcije u generalnoj upotrebi

Najčešće korištena radijalna bazna funkcija je višekadrična, prvi je koristio Hardy oko 1970. Funkcionalni oblik je:

$$\phi(r) = (r^2 + {r_0}^2)^{1/2}$$

gde je r_0 proizvoljan faktor skaliranja. Za višekadrične forme važi da su manje osjetljive na izbor r_0 nego neke druge funkcionalne forme.

Uopšteno, i za višekvadrične i za druge funkcije, r_0 bi trebalo da bude veće od tipičnog razdvajanja tačaka, ali manje od „vanjske skale“ ili broja dimenzija funkcije koja se interpolira. Tačnost interpolacije se može razlikovati u nekoliko redova veličine u zavisnosti od izbora r_0 tako da je svakako vrijedno eksperimentisanja. Jedan od načina za eksperimentiranje je da

se izgradi RBF interpolator koji izostavlja jednu tačku te zatim mjeri greška interpolacije na izostavljenoj točki.

Inverzna višekadrična

$$\phi(r) = (r^2 + {r_0}^2)^{-1/2}$$

daje rezultate koji su usporedivi s višekadričnom formom, ponekad i bolje.

Može se činiti čudnim da funkcija i njena inverzna rade jednako dobro. Objašnjenje je da je ono što je zaista bitno glatkoća i određena svojstva funkcije Fourierove transformacije koja se ne razlikuju mnogo između višekadrične forme i njenog reciprociteta.

thin-plate splajn radijalna bazna funkcija:

$$\phi(r) = r^2 log(r/r_0)$$

uz pretpostavljenu graničnu vrijednost $\phi(0) = 0$. Ova funkcija ima određeno fizičko opravdanje u problemu minimizacije energije povezane sa savijanjem tanke elastične ploče. Međutim, nema naznaka da je općenito bolja od bilo kojeg od prethodno navedenih oblika.

Gausova radijalna bazna funkcija:

$$\phi(r) = exp(-1/2r^2/{r_0}^2)$$

Tačnost interpolacije pomoću Gaussovih baznih funkcija može biti vrlo osjetljiva na r_0, i često se izbjegava iz tog razloga. Međutim, za glatke funkcije i uz optimalni r_0 može se postići vrlo visoka preciznost.

Razmatrane funkcije implementirane su u kod:

```
class RBF_multiquadric : RBF_fn {
    double r02;
public:
    RBF_multiquadric(double scale=1.) : r02(SQR(scale)) {}
    double rbf(double r) { return sqrt(SQR(r)+r02); }
};
```

Program 4.2: Splajn 2D interpolacija - C++ kod

```
class RBF_thinplate : RBF_fn {
    double r0;
public:
    RBF_thinplate(double scale=1.) : r0(scale) {}
    double rbf(double r) { return r <= 0. ? 0. : SQR(r)*log(r/r0); }
};
```

Program 4.3: Splajn 2D interpolacija - C++ kod

```
class RBF_gauss : RBF_fn {
    double r0;
public:
    RBF_gauss(double scale=1.) : r0(scale) {}
    double rbf(double r) { return exp(-0.5*SQR(r/r0)); }
};
```

Program 4.4: Splajn 2D interpolacija - C++ kod

```
1
2 class RBF_inversemultiquadric : RBF_fn {
3     double r02;
4 public:
5     RBF_inversemultiquadric(double scale=1.) : r02(SQR(scale)) {}
6     double rbf(double r) { return 1./sqrt(SQR(r)+r02); }
7 };
```

Program 4.5: Splajn 2D interpolacija - C++ kod

4.3 Shepardova interpolacija

Interesantan poseban slučaj interpolacije normalizirane radijalne bazne funkcije $\phi(r)$ se javlja ako funkcija $\phi(r)$ ide u beskonačnost dok r teži nuli i konačna je za r > 0. U tom slučaju je lako vidjeti da su težine ω_i jednake odgovarajućim funkcijskim vrijednostima y_i, a formula za interpolaciju je jednostavna:

$$y(x) = \sum_{i=0}^{N-1} y_i \phi(|x - x_i|) / \sum_{i=0}^{N-1} \phi(|x - x_i|)$$

Nije potrebno rješenje početnih linearnih jednadžbi. Početna izračunavanja su zanemarljiva, dok su izračunavanja za svaku interpolaciju O(N) podnošljiva čak i za vrlo veliko N.

Shepard je predložio jednostavnu funkciju:

$$\phi(r) = r^{-p}$$

Gdje je 1 < p <= 3, kao i neke složenije funkcije sa različitim eksponentima u unutrašnjem i spoljašnjem području. Može se vidjeti šta se dešava je u osnovi interpolacija najbližim težinskim prosjekom, pri čemu bliže tačke doprinose više od udaljenih tačaka.

Shepardova interpolacija je rijetko tako tačna kao dobro podešena primjena osnovnih radijalnih funkcija. S druge strane je jednostavna i brza.

```
1
2 class Shep_interp {
3     int dim, n;
4     const vector<vector<double> > &pts;
5     const vector<double> &vals;
6     double pneg;
7 public:
8     Shep_interp(vector<vector<double> > &ptss, vector<double> &valss
     , double p=2.)
9     : dim(ptss[0].size()), n(ptss.size()) , pts(ptss), vals(valss),
     pneg(-p) {}
10
11     double interp(vector<double> &pt) {
12         double r, w, sum=0., sumw=0.;
13         if (pt.size() != dim) throw("RBF_interp bad pt size");
14         for (int i=0; i<n; i++) {
15             if ((r=rad(&pt[0],&pts[i][0])) == 0.) return vals[i];
16             sum += (w = pow(r,pneg));
17             sumw += w*vals[i];
```

```
18          }
19          return sumw/sum;
20      }
21
22      double rad(const double *p1, const double *p2) {
23          double sum = 0.;
24          for (int i=0;i<dim;i++) sum += SQR(p1[i]-p2[i]);
25          return sqrt(sum);
26      }
27  };
```

Program 4.6: Splajn 2D interpolacija - C++ kod

4.4 Interpolacija po Krigingu

Kriging je tehnika nazvana po južnoafričkom inženjeru rudarstva D.G. Krige. Tehnika je u suštini oblik linearne predikcije, poznata kao Gauss-Markovljeva procijena ili proces Gaussijan regresije.

Kriging može biti metoda interpolacije ili metoda fitinga. Razlika je u tome da li funkcija fitinga/interpolacije ide tačno kroz sve ulazne tačke (interpolacija), ili dozvoljava da greške mjerenja specificiraju a zatim "izglade" da bi se dobio statistički bolji prediktor koji ne prolazi kroz tačke podataka. Ovdje ćemo razmotriti samo slučaj interpolacije.

Da bi se koristila Kriging metod interpolacije, potrebno je procijeniti srednju kvadratnu varijaciju zadane funkcije $y(x)$ kao funkciju ofseta vrijednosti r što se naziva **variogram**,

$$v(r) \sim \frac{1}{2}\left\langle [y(x+r)-y(x)]^2 \right\rangle$$

gde je prosjek po svim x sa fiksnim r. Interpolacija čak i vrlo grube variogramske procjene rade dobro, u kodu će biti definisaa procedura za procjenu $v(r)$ od zadatih tačaka x_i i vrijednost funkcije u tim tačkama $y_i = y(x_i), i = 0,...,N-1$.

Neka v_{ij} označava $v(|x_i - x_j|)$, gde su i i j zadate tačke, i neka v_{*j} označava $v(|x_* - x_j|)$, x_* tačku u kojoj se želi interpolirati vrijednost $y(x_*)$. Ako se definišu dva vektora dužine N + 1,

$$Y = (y_0, y_1, ..., y_{N-1}, 0)$$

$$V_* = (v_{*1}, v_{*2}, ..., v_{*,N-1}, 1)$$

i simetrična matricu dimenzija (N + 1) x (N + 1),

$$V = \begin{bmatrix} v_{00} & v_{01} & \dots & v_{0,N-1} & 1 \\ v_{10} & v_{11} & \dots & v_{1,N-1} & 1 \\ \dots & \dots & \dots & \dots & \dots \\ v_{N-1,0} & v_{N-1,1} & \dots & v_{N-1,N-1} & 1 \\ 1 & 1 & \dots & 1 & 0 \end{bmatrix}$$

Tada je procjena prema Kiring interpolaciji $\hat{y}_* \approx y(x_*)$ data sa

$$\hat{y}_* = V_* \bullet V_{-1} \bullet Y$$

i njena varijanca sa

$$Var(\hat{y}_*) = V_* \bullet V_{-1} \bullet V_*$$

Kada se jednom izračuna LU dekompoziciju V, a zatim se povrati jednom, zbog dobijanja vektora $V_{-1} \bullet Y$, onda pojedinačna interpolacije košta O(N): Izračunava se vektor V_* i uzima dot produkt. S druge strane, svako računanje varijanse zahtijeva $O(N^2)$.

Svrha dodatnog reda i kolone u V, i dodatnih poslednjih atributa u V_* i Y, je da se automatski izračuna i ispravi odgovarajući težinski prosjek podataka.

Implementacija prethodnih jednačina: Konstruktor radi inicijalni posao, dok dvije preklopljene *interp* metode izračunavaju interpoliranu vrijednost ili vrijednost i standardnu devijaciju (kvadratni korijen varijance). Neka opcijonalni argument err bude postavljen na podrazumijevanu vrijednost NULL.

```
template<class T>
class Krig {
    const vector<vector<double> > &x;
    const T &vgram;
    int ndim, npt;
    double lastval, lasterr;
    vector<double> y,dstar,vstar,yvi;
    vector<vector<double> > v;
    LUdcmp *vi;
public:
    Krig(vector<vector<double> > &xx, vector<double> &yy, T &vargram
    , const double *err=NULL): x(xx),vgram(vargram),npt(xx.size()),
    ndim(xx[0].size()),dstar(npt+1),
    vstar(npt+1),v(npt+1,npt+1),y(npt+1),yvi(npt+1) {
        int i,j;
        for (i=0;i<npt;i++) {
            y[i] = yy[i];
            for (j=i;j<npt;j++) {
                v[i][j] = v[j][i] = vgram(rdist(&x[i][0],&x[j][0]));
            }
            v[i][npt] = v[npt][i] = 1.;
        }
        v[npt][npt] = y[npt] = 0.;
        if (err) for (i=0;i<npt;i++) v[i][i] -= SQR(err[i]);
        vi = new LUdcmp(v);
        vi->solve(y,yvi);
    }
    ~Krig() { delete vi; }

    double interp(vector<double> &xstar) {
        int i;
        for (i=0;i<npt;i++) vstar[i] = vgram(rdist(&xstar[0],&x[i
    ][0]));
        vstar[npt] = 1.;
        lastval = 0.;
        for (i=0;i<=npt;i++) lastval += yvi[i]*vstar[i];
        return lastval;
    }

    double interp(vector<double> &xstar, double &esterr) {
        lastval = interp(xstar);
        vi->solve(vstar,dstar);
        lasterr = 0;
```

```
41          for (int i=0;i<=npt;i++) lasterr += dstar[i]*vstar[i];
42          esterr = lasterr = sqrt(max(0.,lasterr));
43          return lastval;
44      }
45
46      double rdist(const double *x1, const double *x2) {
47          double d=0.;
48          for (int i=0;i<ndim;i++) d += SQR(x1[i]-x2[i]);
49          return sqrt(d);
50      }
51  };
```

Program 4.7: Splajn 2D interpolacija - C++ kod

Argument konstruktora *vgram*, variogramska funkcija, može biti ili funkcija ili funktor. Za interpolaciju, se može koristiti *Powvargram* objekt koji odgovara jednostavnom modelu

$$v(r) = \alpha r^{\beta}$$

gdje se β smatra fiksnim a α se postavlja pomoću najmanjih kvadrata nad svim parovima tačaka i i j. Vrijednost β bi trebala biti u rasponu $1 \leq \beta < 2$. Dobar opšti izbor je 1.5, ali za funkcije sa linearnim rastom, se može eksperimentisati sa vrijednostima koje su blizu gornje granice.

```
1  class Powvargram {
2      double alph, bet, nugsq;
3  public:
4      Powvargram(vector<vector<double> > &x, vector<double> &y, const
       double beta=1.5, const double nug=0.)
5      : bet(beta), nugsq(nug*nug) {
6          int i,j,k,npt=x.size(),ndim=x[0].size();
7          double rb,num=0.,denom=0.;
8          for (i=0;i<npt;i++) for (j=i+1;j<npt;j++) {
9              rb = 0.;
10             for (k=0;k<ndim;k++) rb += SQR(x[i][k]-x[j][k]);
11             rb = pow(rb,0.5*beta);
12             num += rb*(0.5*SQR(y[i]-y[j]) - nugsq);
13             denom += SQR(rb);
14         }
15         alph = num/denom;
16     }
17
18     double operator() (const double r) const {return nugsq+alph*pow(
       r,bet);}
19 };
```

Program 4.8: Splajn 2D interpolacija - C++ kod

Treba biti svjestan da su interpolirane vrijednosti prilično neosjetljive na variogramski model, procijenjene greške su prilično osjetljive. Stoga treba uzeti u obzir procjene grešake samo kao red veličine.

4.5 Interpolacija krivih u više dimenzija

Dat je uređen skup N tačaka u n dimenzija koje leže na jednodimenzionalnoj krivulji $x_0, \ldots, x_{N-1}$, i potrebno je interpolirati vrijednosti duž krivulje. Dva slučaja koja se razlikuju su: (i) Krivulja

je otvorena kriva, tako da x_0 i x_{N-1} predstavljaju krajnje tačke. (ii) Krivulja je zatvorena kriva, tako da postoji segment krive koji povezuje x_{N-1} sa x_0

Jednostavno rješenje je koristiti definisane metode, prvo aproksimirati udaljenost duž krive sumom pravih između zadatih tačaka, a zatim konstruirati interpolacije splajna za svaku od koordinata 0, ..., n-1, kao funkciju tog parametra. S obzirom da izvod bilo koje pojedinačne koordinate u odnosu na dužinu luka ne može biti veći od 1, garantirano je dobro izračunavanje splajn interpolacije.

```
1
2  class Curve_interp {
3      int dim, n, in;
4      bool cls;
5      vector<vector<double> > pts;
6      vector<double> s;
7      vector<double> ans;
8      vector<Spline_interp*> srp;
9  public:
10     Curve_interp(vector<vector<double> > &ptsin, bool close=0): n(
       ptsin.size()), dim(ptsin[0].size()), in(close ? 2*n : n),cls(
       close) {
11         ans.resize(dim);
12         s.resize(in);
13         pts.resize(dim);
14         for(int i = 0; i < dim; i++) pts[0].resize(in);
15
16         int i,ii,im,j,ofs;
17         double ss,soff,db,de;
18         ofs = close ? n/2 : 0;
19         s[0] = 0.;
20         for (i=0;i<in;i++) {
21             ii = (i-ofs+n) % n;
22             im = (ii-1+n) % n;
23             for (j=0;j<dim;j++) pts[j][i] = ptsin[ii][j];
24             if (i>0) {
25                 s[i] = s[i-1] + rad(&ptsin[ii][0],&ptsin[im][0]);
26                 if (s[i] == s[i-1]) throw("error in Curve_interp");
27             }
28         }
29         ss = close ? s[ofs+n]-s[ofs] : s[n-1]-s[0];
30         soff = s[ofs];
31         for (i=0;i<in;i++) s[i] = (s[i]-soff)/ss;
32         for (j=0;j<dim;j++) {
33             db = in < 4 ? 1.e99 : fprime(&s[0],&pts[j][0],1);
34             de = in < 4 ? 1.e99 : fprime(&s[in-1],&pts[j][in-1],-1);
35             srp.push_back(new Spline_interp(s,&pts[j][0],db,de));
36         }
37     }
38     ~Curve_interp() {for (int j=0;j<dim;j++) delete srp[j];}
39
40     vector<double> &interp(double t) {
41         if (cls) t = t - floor(t);
42         for (int j=0;j<dim;j++) ans[j] = (*srp[j]).interp(t);
43         return ans;
44     }
45
46     double fprime(double *x, double *y, int pm) {
47         double s1 = x[0]-x[pm*1], s2 = x[0]-x[pm*2], s3 = x[0]-x[pm
```

```
    *3],s12 = s1-s2, s13 = s1-s3, s23 = s2-s3;
48          return -(s1*s2/(s13*s23*s3))*y[pm*3]+(s1*s3/(s12*s2*s23))*y[
    pm*2]-(s2*s3/(s1*s12*s13))*y[pm*1]+(1./s1+1./s2+1./s3)*y[0];
49      }
50
51      double rad(const double *p1, const double *p2) {
52          double sum = 0.;
53          for (int i=0;i<dim;i++) sum += SQR(p1[i]-p2[i]);
54          return sqrt(sum);
55      }
56
57  };
```

Program 4.9: Splajn 2D interpolacija - C++ kod

Procedura *fprime* procjenjuje izvod funkcije na apscisi x_0 koristeći četiri uzastopna para $(x_0,y_0),\ldots,(x_3,y_3)$. Formula se lako dobija proširenjem pomoću stepenovanog reda:

$$y_0' = -C_0 y_0 + C_1 y_1 - C_2 y_2 + C_3 y_3$$

gdje je

$$C_0 = \frac{1}{s_1} + \frac{1}{s_2} + \frac{1}{s_3}$$

$$C_1 = \frac{s_2 s_3}{s_1(s_2 - s_1)(s_3 - s_1)}$$

$$C_2 = \frac{s_1 s_3}{(s_2 - s_1)s_2(s_3 - s_2)}$$

$$C_3 = \frac{s_1 s_2}{(s_3 - s_1)(s_3 - s_2)s_3}$$

sa

$$s_1 \equiv x_1 - x_0$$

$$s_2 \equiv x_2 - x_0$$

$$s_3 \equiv x_3 - x_0$$

Poglavlje 5

Primjena visedimenzionalne interpolacije

5.1 Primjena višedimenzionalne interpolacije u procjeni performansi

Predviđanje multidimenzionalnih prostornih procesa važan je problem u mnogim područjima istraživanja, kao na primjer u području geologije, praćenja okoliša, meteorologije, tržišta nekretnina i poljoprivrede. U mnogim slučajevima postoji samo ograničen broj tačaka poznatih podataka.

Na osnovu toga, poželjno je uzeti u obzir sve raspoložive podatke. Tipičan primjer je mapiranje zagađenja zraka. Predviđanja se mogu poboljšati korištenjem udaljenosti od cesta i mjerenja drugih zagađivača. Multidimenzionalno interpolacijsko modeliranje, prvo je korišteno za poboljšanje predviđanja Zemljinog gravitacijskog polja korištenjem podataka o mjerenjima vjetra, koje je napravio Lev Gandin 1963.

Cokriging modeli su učinkoviti, ali zahtijevaju određene ograničavajuće pretpostavke, osobito pretpostavke o normalnosti podataka i stacionarnosti. Te su pretpostavke često nerealne, ali služe kao blokovi za sofisticiranije modele. U proširenju ArcGIS Geostatistical Analyst, jedna, dvije ili tri varijable mogu se koristiti kao sekundarne varijable za izradu točnijih predviđanja primarne varijable na neuzorkovanom mjestu.

5.1.1 Procjena podataka o gasu i nafti

5.2 Primjena multidimenzionalne interpolacije na nehomogene spužvaste kosti

5.3 Primjena višedimenzionalne interpolacije u procesiranju slike

Zaključak

Preporučuje se da se poglavlje "Uvod" i "Zaključak", te odgovarajuće sekcije i podsekcije ne numerišu. Ovo poglavlje bi trebalo na izvjestan način objediniti sve "kraće" zaključke date na kraju pojedinih poglavlja.

Ostvareni ciljevi završnog rada

U ovoj sekciji je potrebno dati jasan sumarni pregled obavljenih istraživanja i dobijenih rezultata. Rezultati trebaju biti struktuirani i prikazani prema okvirima i ciljevima postavljenim u uvodnom poglavlju. Potrebno je i provesti poredjenja dobivenih rezultata sa literaturom navedenom u uvodnom poglavlju, te dati diskusiju kako se dobijeni rezultati uklapaju, potvrdjuju, nadopunjuju ili su kontradiktorni onim koji su prikazani u uvodnom poglavlju.

Prilozi

Prilog A

Korištenje funkcija u Tex-u

Sadržaji koji se mogu uključiti u Priloge su: izvođenje jednačina i formula, detalji važnijih softverskih programa, razne tabele i dijagrami, karkateristike i performanse ili opisi opreme i komponenti koje su korištene u disertaciji/radu. Mogu se takodjer uključiti konstrukcioni crteži ili električne sheme.

U ovom prilogu prikazane su neke od funkcije koje se mogu koristiti prilikom oblikovanja rada i prikaza rezultata istraživanja korištenjem LaTeXa.

A.1 Matematički izraz

Primjer matematičke formule prikazan je izrazom

$$T : \mathbf{x}_B \mapsto \mathbf{x}_A \Leftrightarrow T(\mathbf{x}_B) = \mathbf{x}_A. \tag{A.1}$$

Matematičke relacije se u LaTeXrazvojnom okruženju automatski numeriraju. Međutim, da bi se pojedina relacija (slika, tabela) referencirala u tekstu, potrebno je da se svakoj relaciji (slici, tabeli) dodijeli pogodna labela npr. `\label{MojaRelacija}`, a potom referencira u .tex fajlu sa `\ref{MojaRelacija}`. Na taj način će se LaTeXpobrinuti za odgovarajuće kros-referenciranje.

A.2 Slika

Slika **??** služi kao primjer uključivanja slike u tekst. Relacije, slike i tabele se automatski numeriraju u LaTeXu, i to sa oznakom brojpoglavlja.brojslike (npr. u Poglavlju 3 se numeriraju sa 3.1, 3.2, ... neovisno od toga u kojoj sekciji ili podsekciji se nalaze).

A.3 Tabela

Formiranje tabele prikazano je na primjeru u Tabeli A.1. Za razliku od naslova slika, naslov tabela stoji iznad odgovarajućih tabela u tekstu.

A.4 *Landscape*

Postavljanja stranice u prikaz *landscape* prikazano je umetanjem

Tabela A.1: Naslov tabele

Oznaka reda	Kolona 1	Kolona 2	Kolona 3
red 1	1	2	3
red 2	3	2	1
red 3	$E = mc^2$	2	3

A.5 Indeks pojmova i Popis oznaka

Ukoliko je u radu neophodno uvesti i indeks, odnosno popis oznaka, onda se to radi na sljedeći način. Prilikom definiranja indeksa koristi se `\index{ime}`. Npr. `\index{uključivanja slike}`.

Kod dodavanja pojmova u Popis oznaka u .tex fajlu se koristi `\nomenclature{simbol}{opis}`, npr. `\nomenclature{ETF}{Elektrotehnički fakultet}`. Generiranje indeksa i Popisa oznaka se pravi korištenjem naredbi `\makeindex` i `\makenomenclature` u preambli, odnosno `\printindex` i `\printnomenclature` na mjestu generiranja popisa. Osim toga, potrebno je i kompajlirati dokument sa MakeIndex.

A.6 Korištenje literature

Popis literature navodi se na kraju rada. Da bi uz LaTeXefikasno koristila literatura, potrebno je da se generira fajl sa bibliografskim jedinicama. Fajl `literatura.bib` je sastavni dio ovog rada, i može poslužiti kao primjer kako se pišu pojedine bibliografske jedinice. Svaki unos (referenca) sadrži labelu na tu referencu, putem koje se bilo gdje u radu može citirati npr. sa `\cite{Hajn01}` .

Dobar trik za popunjavanje bibliografskih unosa u .bib fajlu je korištenje Google Scholara `https://scholar.google.com/`. Osim što je baza naučnih radova, Google Scholar omogućava i kopiranje zapisa referenci na ispravan način. Podržani su svi najpopularniji formati citiranja (MLA, Chicago, Harvard itd.), kao što se vidi na Slici **??**. Osim toga, klikom na dugme "BibTeX", moguće je izabrati i zapis reference razumljive razvojnom okruženju LaTeX, a nakon toga je jednostavno kopirati u bibliografski fajl `literatura.bib` (vidjeti Sliku **??**).

Primjeri navođenja literature su knjiga [**?**], poglavlje u knjizi [**?**], članak objavljen u časopisu [**?**], članak objavljen na konferenciji [**?**], doktorski rad [**?**], Internetski izvor [**?**] te različite druge publikacije [**?**]. Stil navođenja literature temelji se na stilu razvijenom za IEEE časopise i konferencije.

A.7 Programski kodovi

Programski kodovi se LaTeXu navode korištenjem okruženja `lstlisting`. Primjer koda je dat ispod.

```
1 // program u C++
2 #include <iostream>
3
```

```
4 int main ()
5 {
6   std::cout << "Dobar Dan! ";
7   std::cout << "Prvi program u C++";
8 }
```

Program A.1: Primjer programa

Indeks pojmova

uključivanje slike, 25